

Bash Interview Questions — Networking & DevOps

bash-learning curriculum

Common questions asked at FAANG, cloud providers, SRE / DevOps /
NetOps roles

How to use this document

This is a curated set of bash / shell questions commonly asked in screening and onsite interviews for **DevOps, SRE, Platform, Network Engineering, and Cloud Infrastructure** roles. Questions are grouped by theme and rated:

- **[E]** Easy — phone-screen / basic competence
- **[M]** Medium — most onsite questions live here
- **[H]** Hard — senior / staff or stress-test follow-ups

Each entry includes the question, a concise expected answer, and (where useful) a runnable one-liner or a “watch out for” note.

Table of Contents

1. Fundamentals & Quoting
2. Control Flow & Functions
3. Text Processing (grep / sed / awk)
4. File System & Permissions
5. Processes, Signals & Job Control
6. Networking Commands & Diagnostics
7. DevOps Scenarios (logs, deploys, cron, systemd)
8. Coding Challenges (live coding round)
9. Debugging & Troubleshooting
10. Best Practices & Code Review Questions
11. Rapid-fire round (one-line answers)

1. Fundamentals & Quoting

Q1.1 [E] What is a shebang and why does it matter?

The first line `#!/usr/bin/env bash` tells the kernel which interpreter to use. `/usr/bin/env bash` is preferred over `/bin/bash` for portability — it finds bash on the user’s PATH, which matters on macOS where Homebrew bash lives at `/opt/homebrew/bin/bash`.

Q1.2 [E] Difference between `$var`, `"$var"`, and `'$var'`?

- `$var` → expands and is subject to word splitting + globbing (dangerous if value has spaces).
- `"$var"` → expands but no splitting (the **safe** default).
- `'$var'` → literal text `$var`, no expansion.

Q1.3 [E] Difference between `"$@"` and `"$*"`?

Both expand to all positional parameters. "\$@" expands to **separate words** (" \$1 " " \$2 " ...), "\$*" joins them into a **single string** separated by the first character of IFS. Use "\$@" 99% of the time.

Q1.4 [M] What does set -euo pipefail do?

- -e exit immediately on any non-zero status
- -u error on referencing an unset variable
- -o pipefail a pipeline returns the rightmost non-zero status (not just the last command's)

Pitfall: -e does **not** trigger inside if, &&, ||, or ! contexts.

Q1.5 [M] What is the difference between [, [[, and ((?

- [] → external test command, POSIX-portable, word-splits unquoted vars
- [[]] → bash builtin, safer, supports =~ regex, &&|| inside
- (()) → arithmetic context: variables don't need \$, supports </> as numeric

Q1.6 [E] What does \$? return?

The exit status of the most recently executed foreground command. 0 is success, anything 1-255 is failure.

Q1.7 [M] What's the difference between \$\$ and \$!?

- \$\$ → PID of the current shell
- \$! → PID of the most recently backgrounded process

Q1.8 [M] Why is for f in \$(ls *.txt) considered bad?

- Word splitting breaks on filenames with spaces / tabs / newlines.
- ls output is for humans, not parsing.

Better: for f in *.txt; do [[-e "\$f"]] || continue; ...; done or find . -name '*.txt' -print0 | xargs -0

Q1.9 [H] What's the difference between source script.sh and ./script.sh?

- ./script.sh runs in a **subshell** — variable changes don't affect the caller.
- source script.sh (or . script.sh) runs in the **current shell** — env changes persist.

Q1.10 [M] What does IFS=\$'\n\t' mean in a script header?

Sets the **Internal Field Separator** to newline + tab (not space). Prevents word splitting from breaking on filenames or values with spaces. Common in the “unofficial bash strict mode”.

2. Control Flow & Functions

Q2.1 [E] Write an if statement that runs only if a file exists and is readable.

```
if [[ -r "$file" ]]; then
    cat "$file"
```

fi

-r already implies existence; combine with -f if you also need “regular file”.

Q2.2 [E] How do you read a file line by line, safely?

```
while IFS= read -r line; do
    echo "$line"
done < "$file"
```

- IFS= preserves leading/trailing whitespace
- -r prevents backslash interpretation
- Read from a redirect, not cat file | (avoids a subshell that loses vars)

Q2.3 [M] What’s wrong with this loop?

```
count=0
cat file | while read -r line; do
    ((count++))
done
echo "$count" # always 0
```

The pipeline creates a **subshell** — the inner count modifies a child process variable; the outer count never changes. Fix: use while read ... done < file (no pipe) or use process substitution.

Q2.4 [M] How do you write a function that returns a string?

You don’t — return only sets the exit status (0-255). Instead, **echo** the value and capture with command substitution:

```
upper() { echo "${1^^}"; }
result=$(upper "hello") # HELLO
```

Q2.5 [M] How do you handle command-line flags?

```
while getopts "hf:v" opt; do
    case $opt in
        h) usage; exit 0 ;;
        f) file=$OPTARG ;;
        v) verbose=1 ;;
        \?) echo "unknown: -$OPTARG" >&2; exit 2 ;;
        :) echo "-$OPTARG needs a value" >&2; exit 2 ;;
    esac
done
shift $((OPTIND - 1))
```

For long options, write a manual while [[\$# -gt 0]] loop.

Q2.6 [H] How do you implement retry with exponential backoff?

```
retry() {
    local max=${1:-5}; shift
    local delay=1
    for ((i=1; i<=max; i++)); do
        "$@" && return 0
        echo "attempt $i failed; sleeping ${delay}s" >&2
        sleep "$delay"
        delay=$((delay * 2))
    done
    return 1
}
retry 5 curl -sf https://example.com
```

3. Text Processing (grep / sed / awk)

Q3.1 [E] How do you count the number of lines containing “ERROR” in a log?

```
grep -c 'ERROR' app.log
```

Q3.2 [E] Find IP addresses in a log file.

```
grep -oE 'b([0-9]{1,3}\.){3}[0-9]{1,3}\b' access.log
```

(Regex is loose — for strict validation use awk with field checks.)

Q3.3 [M] Print the top 10 most-frequent IPs hitting your web server.

```
awk '{print $1}' access.log | sort | uniq -c | sort -rn | head -10
```

Or with one less hop: `awk '{c[$1]++} END {for (i in c) print c[i], i}' access.log | sort -rn | head.`

Q3.4 [M] Replace all occurrences of foo with bar in every .conf file under the current directory, in place.

```
find . -name '*.conf' -type f -exec sed -i 's/foo/bar/g' {} +
```

On macOS (BSD sed): `sed -i ''` (with an explicit empty backup arg).

Q3.5 [M] Print lines 50-60 of a file.

```
sed -n '50,60p' file
# or
awk 'NR>=50 && NR<=60' file
```

Q3.6 [M] Show every 2nd column of a CSV.

```
awk -F',' '{print $2}' data.csv
# or cut -d',' -f2 data.csv
```

Q3.7 [M] Sum the values of column 3 (CSV).

```
awk -F',' '{s += $3} END {print s}' data.csv
```

Q3.8 [M] Show usernames in /etc/passwd with UID >= 1000.

```
awk -F: '$3 >= 1000 {print $1}' /etc/passwd
```

Q3.9 [H] Given an Apache access log, show the busiest hour of the day.

```
awk '{print $4}' access.log \
| cut -d: -f2 \
| sort | uniq -c | sort -rn | head -1
```

\$4 looks like [10/Oct/2024:14:23:01, splitting on : gives the hour.

Q3.10 [H] Replace text only between two markers (BEGIN/END) in a file.

```
sed '/BEGIN/,/END/ s/old/new/g' file
```

Q3.11 [M] Print unique lines while preserving order.

```
awk '!seen[$0]++' file
```

Q3.12 [M] Print the difference between two sorted files (lines in A not in B).

```
comm -23 <(sort A) <(sort B)
```

4. File System & Permissions

Q4.1 [E] What do `chmod 755` and `chmod 644` mean?

- 755 → owner: rwx, group: r-x, others: r-x (executables, directories)
- 644 → owner: rw-, group: r-, others: r- (regular config / data files)

Q4.2 [E] What is the difference between hard and soft (symbolic) links?

- **Hard link** (ln a b) → second directory entry for the same inode. Cannot cross filesystems; survives deleting the original.
- **Symbolic link** (ln -s a b) → pointer file containing a path. Can cross filesystems; **breaks** if the target is removed.

Q4.3 [M] How do you find files larger than 100 MB modified in the last 7 days?

```
find /var/log -type f -size +100M -mtime -7
```

Q4.4 [M] Delete files older than 30 days in /tmp safely.

```
find /tmp -type f -mtime +30 -print0 | xargs -0 -r rm --
```

-print0 + xargs -0 is filename-safe; -r skips empty input.

Q4.5 [M] What does `umask 022` mean?

The “mask” of bits **removed** from default permissions when files are created. 022 → group and others lose write → default file 644, default dir 755.

Q4.6 [H] What’s the difference between `setuid`, `setgid`, and the sticky bit?

- `setuid` (chmod u+s) on executable → runs with **owner’s** privileges
- `setgid` on executable → runs with **group’s** privileges; on directory → new files inherit the directory’s group
- **Sticky bit** (chmod +t) on a dir (e.g. /tmp) → only the file owner can delete files in that directory

Q4.7 [M] How do you copy a directory preserving permissions, owners, and symlinks?

```
cp -a src/ dst/      # GNU  
rsync -a src/ dst/    # cross-platform, preferred for large trees
```

5. Processes, Signals & Job Control

Q5.1 [E] Show all running processes and their parent PIDs.

```
ps -ef          # POSIX
ps auxf        # BSD-style tree
ps -eo pid,ppid,cmd
```

Q5.2 [E] Find the PID of all nginx processes.

```
pgrep -a nginx
# or
ps -eo pid,cmd | grep '[n]ginx'
```

The [n]ginx trick prevents grep from matching itself.

Q5.3 [M] Kill all processes named myapp.

```
pkill myapp      # SIGTERM
pkill -9 myapp   # SIGKILL — last resort
```

Always send SIGTERM first, give the app a few seconds, then SIGKILL if needed.

Q5.4 [M] Difference between SIGTERM, SIGKILL, and SIGHUP?

- SIGTERM (15) → polite “please shut down”; **can** be trapped
- SIGKILL (9) → forceful; **cannot** be trapped, ignored, or blocked
- SIGHUP (1) → originally “terminal hung up”; many daemons reuse it as “reload config”

Q5.5 [M] Run a long command in the background and wait for it.

```
long_task &
pid=$!
# do other work
wait "$pid"
echo "exit was $?"
```

Q5.6 [M] Run N tasks in parallel and wait for all.

```
for url in "${urls[@]}; do
  curl -sf "$url" -o "out-$(basename "$url")" &
done
wait
```

With concurrency limit, use xargs -P or GNU parallel.

Q5.7 [M] What is nohup and what does it do?

nohup cmd & runs cmd immune to SIGHUP (so it survives the controlling terminal closing) and redirects stdout/stderr to nohup.out by default.

Q5.8 [H] Trap Ctrl-C and clean up before exiting.

```
tmp=$(mktemp -d)
cleanup() { rm -rf "$tmp"; }
trap cleanup EXIT
trap 'echo "interrupted"; exit 130' INT
```

trap ... EXIT is the canonical pattern for guaranteed cleanup.

Q5.9 [H] What does disown do?

Removes a job from the shell's job table so it won't receive SIGHUP when the shell exits — useful when you forgot to nohup.

6. Networking Commands & Diagnostics

Q6.1 [E] How do you test if a remote port is open?

```
nc -zv host 443
# or
timeout 3 bash -c '</dev/tcp/host/443' && echo open
```

Q6.2 [E] Show all listening TCP ports and which process owns them.

```
ss -ltnp          # modern (iproute2)
netstat -ltnp     # legacy
lsof -iTCP -sTCP:LISTEN -n -P
```

Q6.3 [E] Look up an A record vs full DNS answer.

```
dig +short example.com A
dig example.com ANY
host example.com
nslookup example.com
```

Q6.4 [M] What's the difference between curl -I, curl -i, and curl -v?

- -I → **HEAD** request only; prints response headers
- -i → include response headers in normal output (full body GET)
- -v → verbose: shows request + response headers, TLS handshake details

Q6.5 [M] Show the response time of an HTTP request.

```
curl -o /dev/null -s -w 'DNS=%{time_namelookup} CONNECT=%{time_connect}
TLS=%{time_appconnect} TTFB=%{time_starttransfer} TOTAL=%
{time_total}\n' https://example.com
```

Q6.6 [M] Show the certificate of an HTTPS site.

```
openssl s_client -servername example.com -connect example.com:443 </dev/null
2>/dev/null \
| openssl x509 -noout -dates -subject -issuer
```

Q6.7 [M] Show the routing table.

```
ip route          # Linux iproute2
route -n          # legacy
netstat -rn       # cross-platform
```

Q6.8 [M] Show ARP cache / neighbor table.

```
ip neigh
```

```
arp -an
```

Q6.9 [M] Capture HTTP traffic on port 80 with tcpdump (no SSL).

```
sudo tcpdump -i any -A -s0 'tcp port 80'
```

Q6.10 [M] What's the difference between ping and mtr?

ping shows round-trip to a destination. mtr (or traceroute) shows **per-hop** loss/latency along the path — far more useful for diagnosing where a network is degraded.

Q6.11 [H] Test latency to every host in a file, in parallel.

```
xargs -P 20 -I{} sh -c 'ping -c1 -W1 {} >/dev/null && echo "{} up" || echo "{} down"' < hosts.txt
```

Q6.12 [H] Show top talkers on the network using ss.

```
ss -tan state established | awk 'NR>1 {split($5,a,"."); print a[1]}' | sort | uniq -c | sort -rn | head
```

Q6.13 [H] You can ping a host but not ssh to it. What do you check?

1. Port: nc -zv host 22 → reachable?
2. Local firewall on target: iptables -L -n / ufw status / firewall-cmd --list-all
3. SSH daemon: systemctl status sshd; ss -ltnp | grep :22
4. SSH logs: journalctl -u sshd -f on the host
5. Selinux/AppArmor blocking sshd
6. Wrong port (custom config) — check /etc/ssh/sshd_config
7. TCP wrappers / /etc/hosts.deny
8. Network path differs from ICMP (routing or middleboxes filtering port 22)

Q6.14 [H] What is the difference between a “tap” and a “tun” interface?

- **tun** → Layer-3 (IP packets) — typical for VPNs (e.g. OpenVPN, WireGuard)
- **tap** → Layer-2 (Ethernet frames) — for bridging, virtual switches

7. DevOps Scenarios (logs, deploys, cron, systemd)

Q7.1 [E] Tail a log and grep for errors live.

```
tail -F app.log | grep --line-buffered -i 'error|fatal'
```

-F follows by **name** so it survives log rotation; --line-buffered flushes per-line so grep doesn't buffer.

Q7.2 [M] You need to find which file is filling up /var/log.

```
du -sh /var/log/* | sort -h  
# or
```

```
du -ah /var/log | sort -h | tail -20
ncdu /var/log # interactive
```

Q7.3 [M] Schedule a job to run every 5 minutes via cron.

```
*/* * * * * /usr/local/bin/job.sh >> /var/log/job.log 2>&1
```

Always redirect stdout+stderr or you'll silently lose output.

Q7.4 [M] Show the last 50 lines of a systemd unit's log.

```
journalctl -u nginx -n 50 --no-pager
journalctl -u nginx -f # follow
journalctl -u nginx --since "1 hour ago"
```

Q7.5 [M] Restart a service and verify it's healthy.

```
systemctl restart nginx
systemctl is-active nginx # active
curl -fs http://localhost/healthz || exit 1
```

Q7.6 [M] Write a script that backs up /etc daily, keeping the last 7 backups.

```
#!/usr/bin/env bash
set -euo pipefail
dst=/backup
ts=$(date +%F)
tar -czf "$dst/etc-$ts.tar.gz" /etc
# Keep newest 7
ls -lt "$dst"/etc-*.tar.gz | tail -n +8 | xargs -r rm --
```

Q7.7 [M] You deploy a new version that crashes immediately. How do you roll back fast?

- Symlink-based deploy: releases/v123/ <- current → just flip the symlink back to the previous release dir, then reload the app.
- Tagged docker image: docker compose up -d app:v122 (or update the tag and re-deploy via your orchestrator).
- Capture this in a one-button rollback script that takes a version arg.

Q7.8 [H] Write a one-liner to find the 5 largest log entries (by file size) older than 7 days under /var/log.

```
find /var/log -type f -mtime +7 -printf '%s\t%p\n' | sort -rn | head -5
```

Q7.9 [H] Your script writes to a logfile. After log rotation, new writes go to nowhere. Why?

Your script still holds an **open file descriptor** to the rotated (and possibly deleted) file. Fixes: - Reopen the file on SIGHUP (trap 'exec >>/var/log/app.log 2>&1' HUP) - Use logger to send to syslog instead - Configure copytruncate in logrotate

Q7.10 [M] Sketch a script that healthchecks N URLs and exits non-zero if any fails.

```
#!/usr/bin/env bash
set -uo pipefail
urls=("$@")
```

```
rc=0
for u in "${urls[@]"; do
    code=$(curl -s -o /dev/null -w '%{http_code}' --max-time 5 "$u")
    [[ "$code" == 2?? ]] || { echo "DOWN $u ($code)" >&2; rc=1; }
done
exit "$rc"
```

8. Coding Challenges (live coding round)

Q8.1 [M] Write a script that prints the top-5 most-used commands from your ~/.bash_history.

```
awk '{print $1}' ~/.bash_history | sort | uniq -c | sort -rn | head -5
```

Q8.2 [M] Reverse a string in pure bash (no rev).

```
reverse() {
    local s=$1 out=""
    for ((i=${#s}-1; i>=0; i--)); do out+="${s:i:1}"; done
    echo "$out"
}
```

Q8.3 [M] Check if a string is a palindrome.

```
is_palindrome() {
    local s=${1,,}
    [[ "$s" == "$(echo "$s" | rev)" ]]
}
```

Q8.4 [M] Print the Nth largest number from a file of integers.

```
sort -rn file | awk -v n="$1" 'NR==n'
```

Q8.5 [M] Given a directory, find duplicate files by content.

```
find . -type f -exec md5sum {} + | sort | awk '
{
    if ($1 == prev) print prev_file, $2
    prev=$1; prev_file=$2
}'
```

(Or use fdupes if allowed.)

Q8.6 [H] Write a script that counts words across all .md files in a tree and prints the 20 most-common words (lowercased, ignoring punctuation).

```
find . -name '*.md' -print0 \
| xargs -0 cat \
| tr '[:upper:]' '[:lower:]' \
| tr -cs '[:alpha:]' '\n' \
| grep -v '^$' \
| sort | uniq -c | sort -rn | head -20
```

Q8.7 [H] Given a CSV with name,age,salary, output the avg salary per age bucket (<30, 30-50, >50).

```
awk -F, 'NR>1 {
```

```

    b = ($2<30 ? "<30" : ($2<=50 ? "30-50" : ">50"))
    sum[b] += $3; n[b]++
}
END { for (k in sum) printf "%-5s %.2f\n", k, sum[k]/n[k] }' data.csv

```

Q8.8 [H] Implement a simple rate limiter: max 5 requests in any 10-second window. Block if exceeded.

```

window=10
max=5
state=$(mktemp)
trap 'rm -f "$state"' EXIT

now=$(date +%s)
# keep only timestamps within window
grep -E '[0-9]+' "$state" 2>/dev/null \
| awk -v now="$now" -v w="$window" '$1 >= now - w' > "$state.tmp"
mv "$state.tmp" "$state"
count=$(wc -l < "$state")
if (( count >= max )); then
    echo "rate-limited"; exit 1
fi
echo "$now" >> "$state"
do_work

```

9. Debugging & Troubleshooting

Q9.1 [E] How do you run a script with tracing to see every command executed?

```

bash -x script.sh
# or inside the script:
set -x # turn on
set +x # turn off

```

Customize the trace prefix: PS4='+
 \${BASH_SOURCE}:\${LINENO}:\${FUNCNAME[0]:-MAIN}: '

Q9.2 [M] You ran a script and nothing happened. How do you debug?

1. bash -n script.sh → syntax-only check
2. bash -x script.sh → trace each command
3. Check \$? after each step interactively
4. Re-run with set -euo pipefail to surface silent failures
5. Run shellcheck script.sh — catches 90% of bugs statically

Q9.3 [M] Script works on Ubuntu but fails on macOS — likely culprits?

- sed -i requires an explicit arg on BSD: sed -i ''
- readlink -f doesn't exist on BSD readlink
- date -d "1 day ago" is GNU-only; BSD uses date -v-1d
- getopt differs (GNU has long options, BSD doesn't)
- Default shell is zsh on newer macOS

Q9.4 [M] Your cron job works manually but fails under cron. Why?

- Different environment: cron has minimal PATH and no shell rc files
- No TTY: anything reading stdin or expecting a terminal will fail
- Different working directory (always start with cd "\$(dirname "\$0")" or pass absolute paths)

- Permissions: cron may run as a different user
- Locale: LC_ALL defaults differ Fix: source your environment explicitly or wrap with bash -lc '...'.

Q9.5 [M] Pipeline exits with non-zero but set -e doesn't trigger. Why?

Without set -o pipefail, only the **last** command in a pipeline determines the pipeline's exit status. A failing producer (curl | jq) won't fail the script. Always combine set -e with set -o pipefail.

Q9.6 [H] Find which process is using a particular file.

```
ls -l /var/log/app.log
lsof /var/log/app.log
```

Q9.7 [H] A bash script consumes 100% CPU. How do you find the hot loop?

- Add set -x and look for the line repeated many times
- strace -p <pid> to see system calls
- pstree -p <pid> for child spawning
- If shelling out a lot, replace external calls (grep, cut) with bash builtins / parameter expansion

10. Best Practices & Code Review Questions

Q10.1 What's wrong with this snippet?

```
file=$1
if [ $file = "" ]; then
    echo "no file"
fi
```

- \$1 and \$file unquoted → breaks on spaces, and breaks the test if empty
- Should use [[]] and -z:

```
if [[ -z "${1:-}" ]]; then echo "no file" >&2; exit 1; fi
```

Q10.2 What's wrong with rm -rf \$DIR/*?

If DIR is empty or unset, this becomes rm -rf /. Always:

```
: "${DIR:?DIR must be set}"
rm -rf -- "${DIR:?}/*"
```

Q10.3 Why is this insecure?

```
eval "cmd $userinput"
```

Anything the user types is executed. Never eval untrusted input. Use arrays and explicit invocation instead.

Q10.4 Name 5 standard items in a production-grade bash script header.

1. #!/usr/bin/env bash shebang
2. set -euo pipefail
3. IFS=\$'\n\t'
4. LC_ALL=C (deterministic sort / regex behaviour)

5. `cd "$(dirname "${BASH_SOURCE[0]}")"` for self-relative paths

Q10.5 When should you NOT use bash?

- | ~200 lines or non-trivial data structures
- Needs unit tests / mocks
- Performance-critical (every external call forks)
- Complex JSON/XML manipulation (use jq for simple cases; otherwise Python)
- Anything involving floating-point math beyond trivial use

Q10.6 What tool would you use in CI to lint bash scripts?

shellcheck. Run it on every script in CI. Treat warnings as errors over time. Bonus tools: shfmt (formatter), bats (testing framework), bashate (style).

Q10.7 How do you make a script both *sourceable* (as a library) and *runnable*?

```
main() { ...; }
if [[ "${BASH_SOURCE[0]}" == "${0}" ]]; then
    main "$@"
fi
```

11. Rapid-fire round (one-line answers)

#	Question	Answer
1	Number argument check	<code>[["\$x" =~ ^[0-9]+\$]]</code>
2	Get script directory	<code>cd "\$(dirname "\${BASH_SOURCE[0]}")"</code>
3	Default value	<code>\${var:-default}</code>
4	Require value or die	<code>\${var:?must be set}</code>
5	Uppercase	<code>\${str^^}</code>
6	Lowercase	<code>\${str,,}</code>
7	Length	<code>\${#str}</code>
8	Substring	<code>\${str:offset:len}</code>
9	Strip prefix	<code>\${str#prefix}</code> (greedy: <code>##</code>)
10	Strip suffix	<code>\${str%suffix}</code> (greedy: <code>%%</code>)
11	Replace first	<code>\${str/old/new}</code>
12	Replace all	<code>\${str//old/new}</code>
13	Array length	<code>\${#arr[@]}</code>
14	All array items (safe)	<code>"\${arr[@]}"</code>
15	Loop array safely	<code>for x in "\${arr[@]}; do ... done</code>
16	Sleep 100 ms	<code>sleep 0.1</code>
17	Current epoch	<code>date +%s</code>
18	ISO timestamp	<code>date -u +%Y-%m-%dT%H:%M:%SZ</code>
19	Make tmp file	<code>f=\$(mktemp)</code>
20	Make tmp dir	<code>d=\$(mktemp -d)</code>
21	Trap cleanup	<code>trap 'rm -rf "\$d"' EXIT</code>
22	Process count of a name	<code>pgrep -c nginx</code>
23	Wait for any bg job	<code>wait -n</code>
24	All bg PIDs	<code>jobs -p</code>

25	TCP port test	nc -zv host port
26	Bash version	\$BASH_VERSION or bash --version
27	Check command exists	command -v jq >/dev/null
28	Redirect both streams	cmd &>file (or >file 2>&1)
29	Pipe stderr too	cmd 2>&1 \ less
30	Silent	cmd >/dev/null 2>&1
31	Last bg PID	\$!
32	Last exit code	\$?
33	This shell's PID	\$\$
34	All args (safe)	"\$@"
35	Argument count	\$#
36	Shift one arg	shift
37	Read with prompt	read -rp "prompt: " var
38	Read with timeout	read -t 5 -p ...
39	Read silent (password)	read -s -p ...
40	Assoc array	declare -A m; m[k]=v

Appendix — What to study the night before

1. **set -euo pipefail** — explain it perfectly; know its gotchas.
2. **Quoting** — be able to defend why every "\$var" is quoted.
3. **[[]] vs []** — pick [[]] and explain why.
4. **Parameter expansion table** — \${var:-x}, \${var:?x}, \${var#x}, \${var%x}, \${var//a/b}.
5. **Reading a file safely** — while IFS= read -r line; do ... done < file.
6. **Top-N pipeline** — ... | sort | uniq -c | sort -rn | head.
7. **awk basics** — -F, \$1, NR, NF, BEGIN/END, summing a column.
8. **Trap-based cleanup** — mktemp + trap '...' EXIT.
9. **getopts skeleton** — be able to write one from memory in 60 seconds.
10. **Networking trio** — nc -zv, ss -ltnp, curl -sIL -w '%{http_code}\n'.

Walk in able to write, on a whiteboard, in under 3 minutes:

```
#!/usr/bin/env bash
set -euo pipefail
IFS=$'\n\t'

usage() { echo "usage: $0 -f FILE [-v]" >&2; exit 2; }

file=""; verbose=0
while getopts ":f:vh" o; do
    case $o in
        f) file=$OPTARG ;;
        v) verbose=1 ;;
        h) usage ;;
        *) usage ;;
    esac
done
[[ -r "$file" ]] || { echo "ERROR: cannot read $file" >&2; exit 1; }

tmp=$(mktemp -d); trap 'rm -rf "$tmp"' EXIT

while IFS= read -r line; do
    (( verbose )) && echo "DEBUG: $line" >&2
    echo "$line"
done < "$file"
```

If you can produce that template under interview pressure and *explain every line*, you will pass any bash screening.

